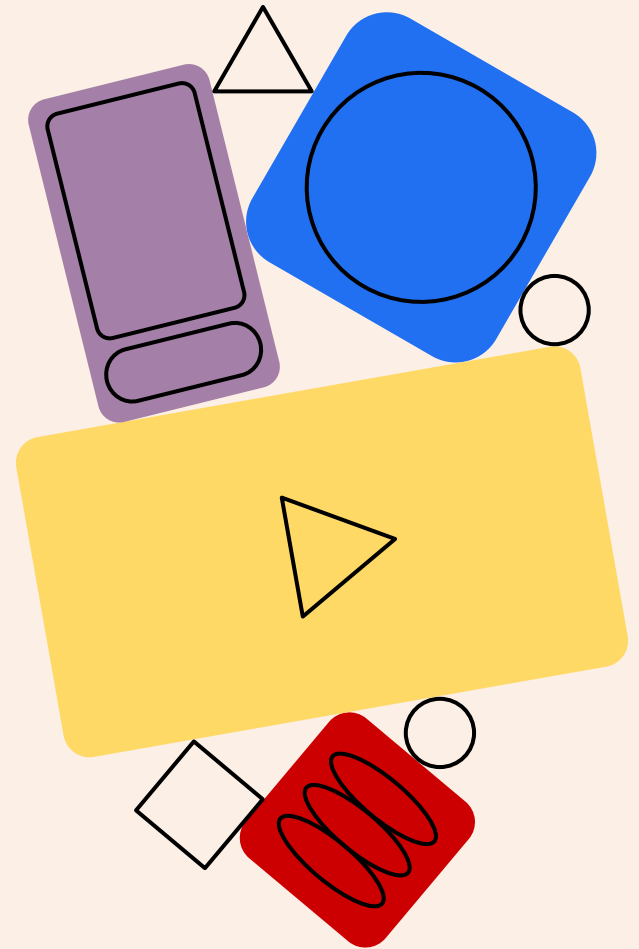


Linear Data Structures - Linked List



Introduction

- An array is the most basic sequence container used to store and access a collection of data. It provides easy and direct access to the individual elements and is supported at the hardware level.
- But arrays are limited in their functionality.
- The Python list, which is also a sequence container, is an abstract sequence type implemented using an array structure.
- It extends the functionality of an array by providing a larger set of operations than the array, and it can automatically adjust in size as items are added or removed.

Introduction

- Python's list class is highly optimized, and often a great choice for storage. But, there are some notable disadvantages:
 - The length of a dynamic array might be longer than the actual number of elements that it stores.
 - Amortized bounds for operations may be unacceptable in real-time systems.
 - Insertions and deletions at interior positions of an array are expensive.
- we introduce a data structure known as a linked list, which provides an alternative to an array-based sequence

Introduction

- An array provides the more centralized representation, with one large chunk of memory capable of accommodating references to many elements.
- A linked list, in contrast, relies on a more distributed representation in which a lightweight object, known as a node, is allocated for each element.
- Each node maintains a reference to its element and one or more references to neighboring nodes in order to collectively represent the linear order of the sequence.

Introduction

- Elements of a linked list cannot be efficiently accessed by a numeric index k , and we cannot tell just by examining a node if it is the second, fifth, or twentieth node in the list.
- However, linked lists avoid the three disadvantages noted above for array-based sequences.

Introduction

Several disadvantages in the use of the array and Python list.

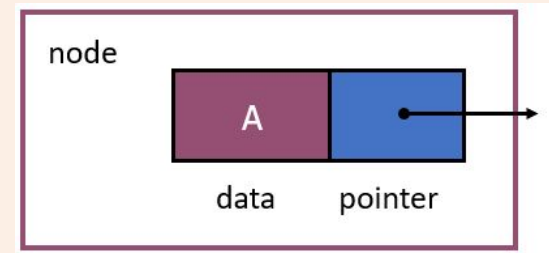
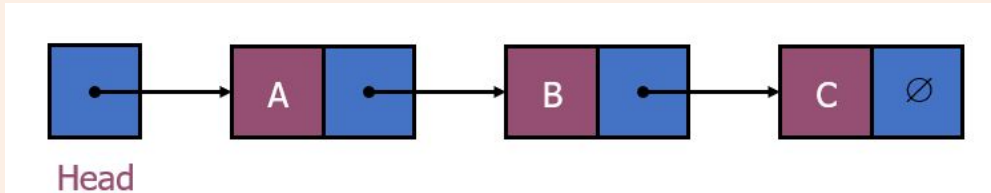
- First, insertion and deletion operations typically require items to be shifted to make room or close a gap.
 - This can be time consuming, especially for large sequences.
- Second, the size of an array is fixed and cannot change. While the Python list does provide for an expandable collection, that expansion does not come without a cost.
 - Since the elements of a Python list are stored in an array, an expansion requires the creation of a new larger array into which the elements of the original array have to be copied.

Introduction

- Finally, the elements of an array are stored in contiguous bytes of memory, no matter the size of the array.
 - Each time an array is created, the program must find and allocate a block of memory large enough to store the entire array.
 - For large arrays, it can be difficult or impossible for the program to locate a block of memory into which the array can be stored.
 - This is especially true in the case of a Python list that grows larger during the execution of a program since each expansion requires ever larger blocks of memory.

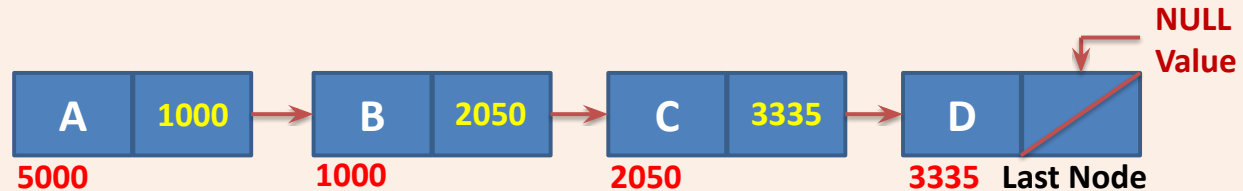
Introduction - Singly Linked Lists

- There are many applications where sequential allocation method is unacceptable because of following characteristics
 - Unpredictable storage requirement
 - Extensive manipulation of stored data
- One method of obtaining the address of node is to store address in computer's main memory, we refer this addressing mode as pointer of link addressing.
- A simple way to represent a linear list is to expand each node to contain a link or pointer to the next node. This representation is called one-way chain or Singly Linked Linear List.



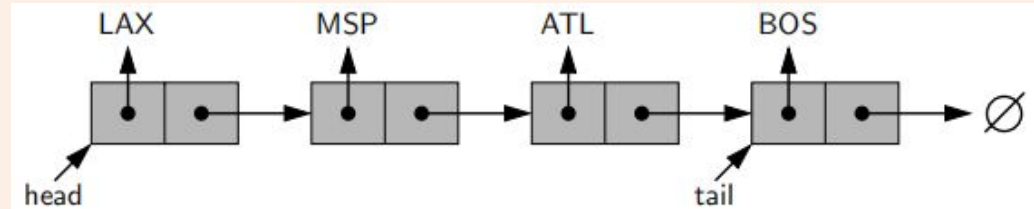
Introduction - Singly Linked Lists

- The linked allocation method of storage can result in both efficient use of computer storage and computer time.
- A linked list is a non-sequential collection of data items.
- Each node is divided into two parts, the first part represents the information of the element and the second part contains the address of the next node.
- The last node of the list does not have successor node, so null value is stored as the address.
- It is possible for a list to have no nodes at all, such a list is called empty list.



Introduction - Singly Linked Lists

- The list is not required to be contiguously present in the memory.
- The node can reside anywhere in the memory and linked together to make a list. This achieves optimized utilization of space.
- List size is limited to the memory size and doesn't need to be declared in advance.
- Empty node can not be present in the linked list.
- A **singly linked list**, in its simplest form, is a collection of nodes that collectively form a linear sequence.
- Each node stores a reference to an object that is an element of the sequence, as well as a reference to the next node of the list



Introduction - Singly Linked Lists

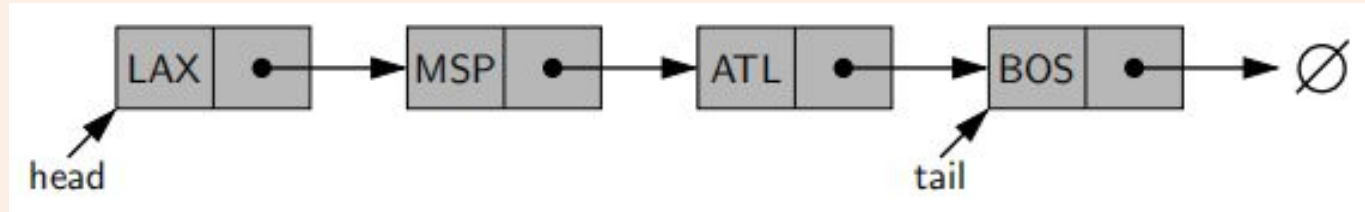
- The first and last node of a linked list are known as the **head** and **tail** of the list, respectively.
- By starting at the head, and moving from one node to another by following each node's next reference, we can reach the tail of the list.
- We can identify the tail as the node having None as its next reference.
- This process is commonly known as traversing the linked list.
- Because the next reference of a node can be viewed as a link or pointer to another node, the process of traversing a list is also known as link hopping or pointer hopping.

Introduction - Singly Linked Lists

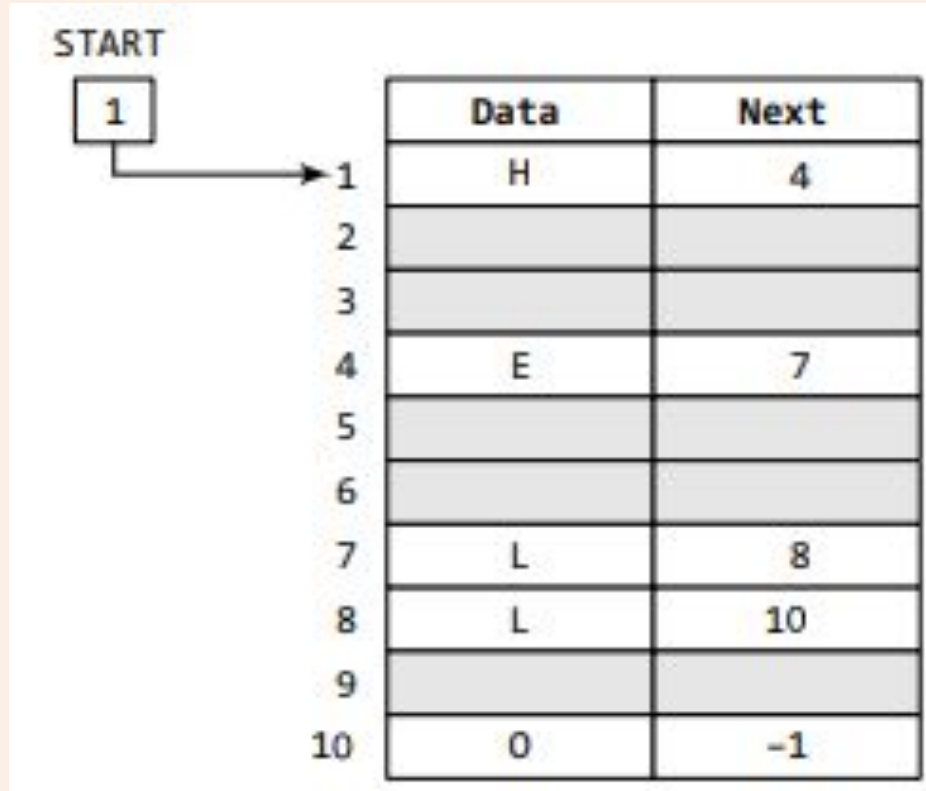
- A linked list's representation in memory relies on the collaboration of many objects.
- Each node is represented as a unique object, with that instance storing a reference to its element and a reference to the next node (or None).
- Another object represents the linked list as a whole.
- Minimally, the linked list instance must keep a reference to the head of the list.
- Without an explicit reference to the head, there would be no way to locate that node (or indirectly, any others).

Introduction - Singly Linked Lists

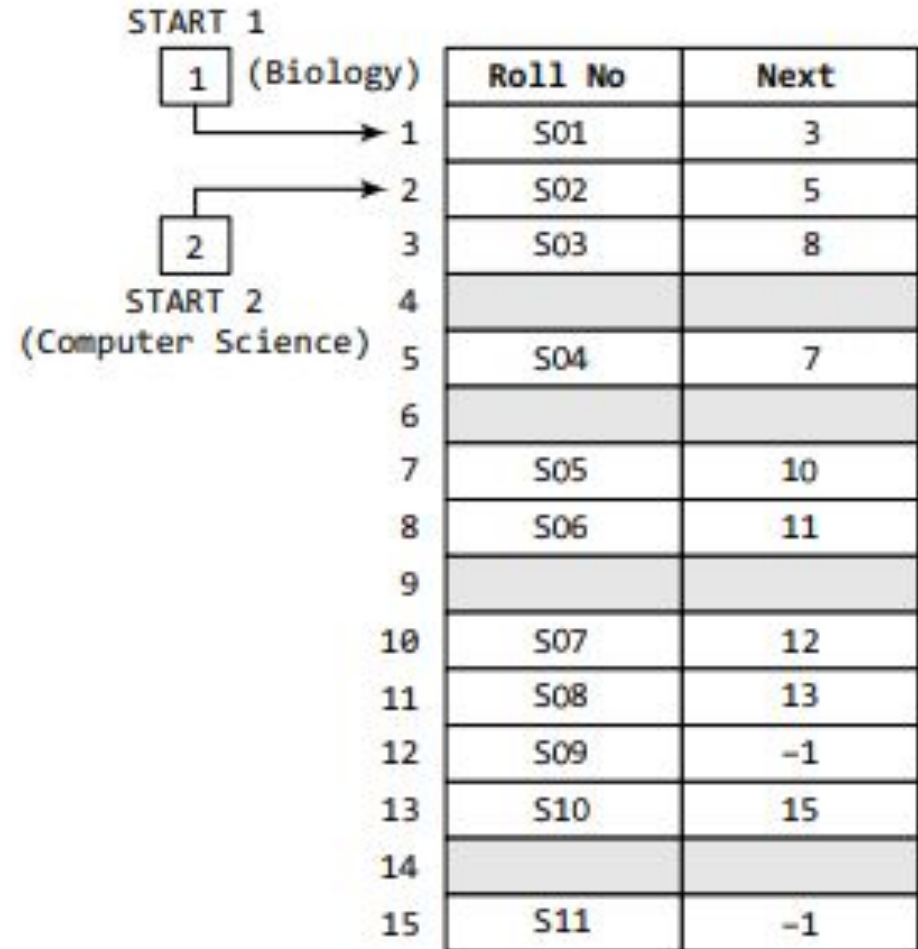
- There is not an absolute need to store a direct reference to the tail of the list, as it could otherwise be located by starting at the head and traversing the rest of the list.
- However, storing an explicit reference to the tail node is a common convenience to avoid such a traversal.
- In similar regard, it is common for the linked list instance to keep a count of the total number of nodes that comprise the list (commonly described as the size of the list), to avoid the need to traverse the list to count the nodes.



Introduction - Singly Linked Lists



Introduction

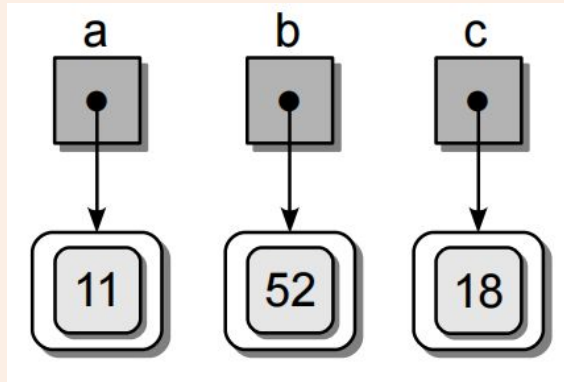


Singly Linked List

- Creation of Node

```
class ListNode :  
    def __init__( self, data ) :  
        self.data = data
```

- We can create several instances of this class, each storing data of our choosing.

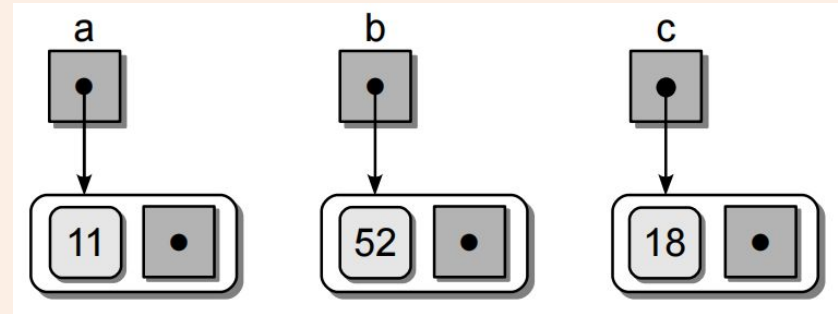


Singly Linked List

- Creation of Node

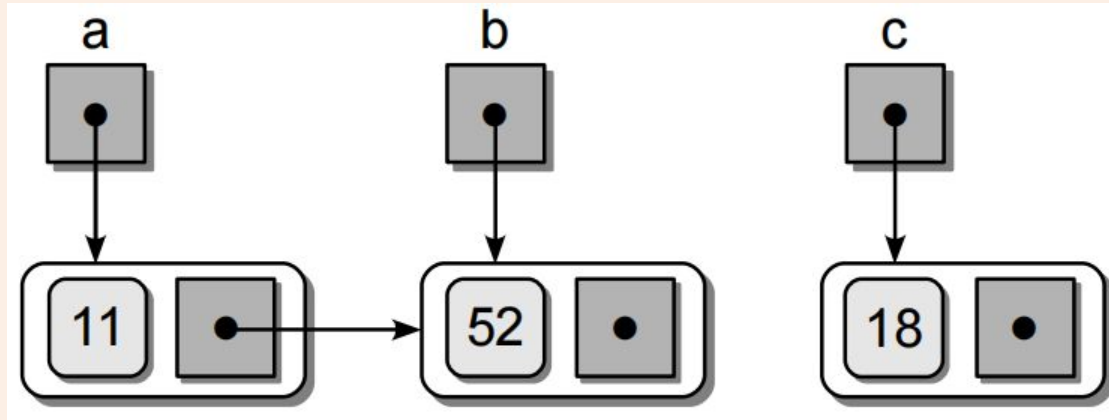
```
class ListNode :  
    def __init__( self, data ) :  
        self.data = data  
        self.next = None
```

- We can create several instances of this class, each storing data of our choosing.
- This time it will have two members.
 - Data and
 - Next reference set to None



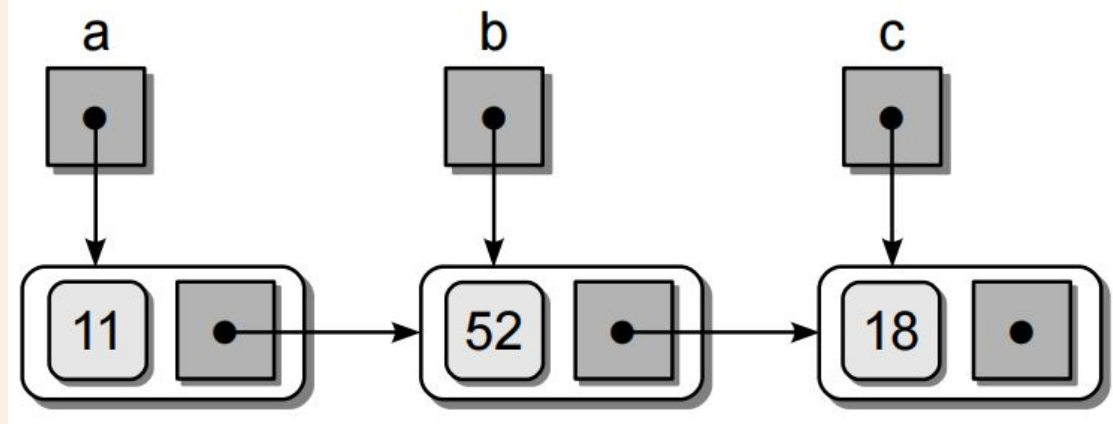
Singly Linked List

- Since the next field can contain a reference to any type of object, we can assign to it a reference to one of the other ListNode objects.
- For example, suppose we assign b to the next field of object a:
`a.next = b`
- which results in object a being linked to object b,



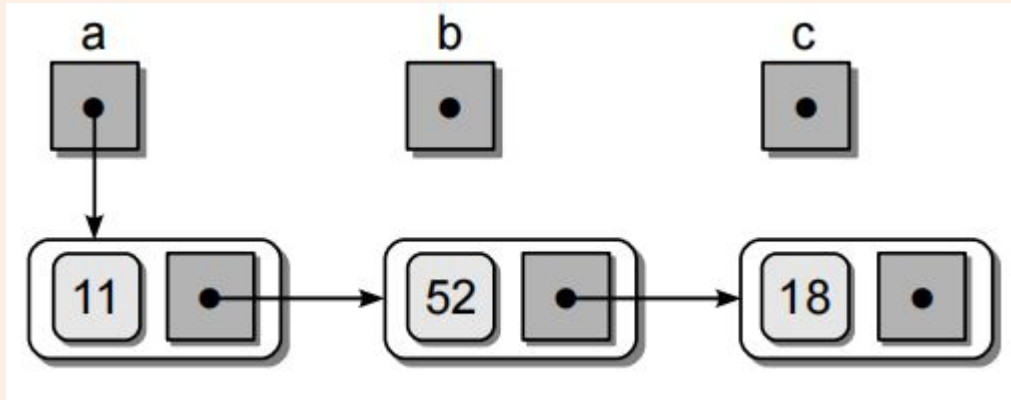
Singly Linked List

- Since the next field can contain a reference to any type of object, we can assign to it a reference to one of the other ListNode objects.
- For example, suppose we assign c to the next field of object b:
`b.next = c`
- which results in a chain of objects,



Singly Linked List

- Lets remove unwanted variables
b = None
c = None
- The result is a linked list structure accessible via variable "a".



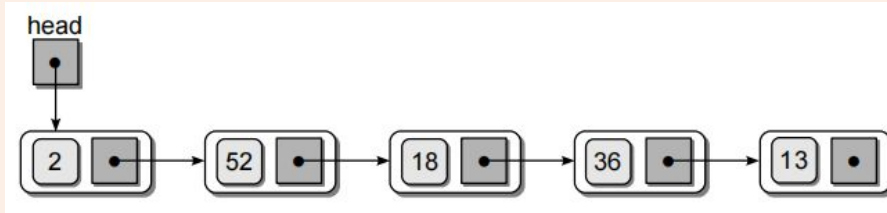
Singly Linked List

- The result is a linked list structure.
- The two objects previously pointed to by b and c are still accessible via a.
- For example, suppose we wanted to print the values of the three objects. We can access the other two objects through the next field of the first object:

```
print( a.data )           # 11
print( a.next.data )      # 52
print( a.next.next.data ) # 18
```

Singly Linked List

- A linked structure contains a collection of objects called **nodes**, each of which contains data and at least one reference or link to another node.
- A linked list is a linked structure in which the nodes are connected in sequence to form a linear list.
- Most nodes in the list have no name and are simply referenced via the link field of the preceding node.
- The first node in the list, however, must be named or referenced by an external variable as it provides an entry point into the linked list.
- This variable is commonly known as the head pointer, or head reference.
- A linked list can also be empty, which is indicated when the head reference is null.

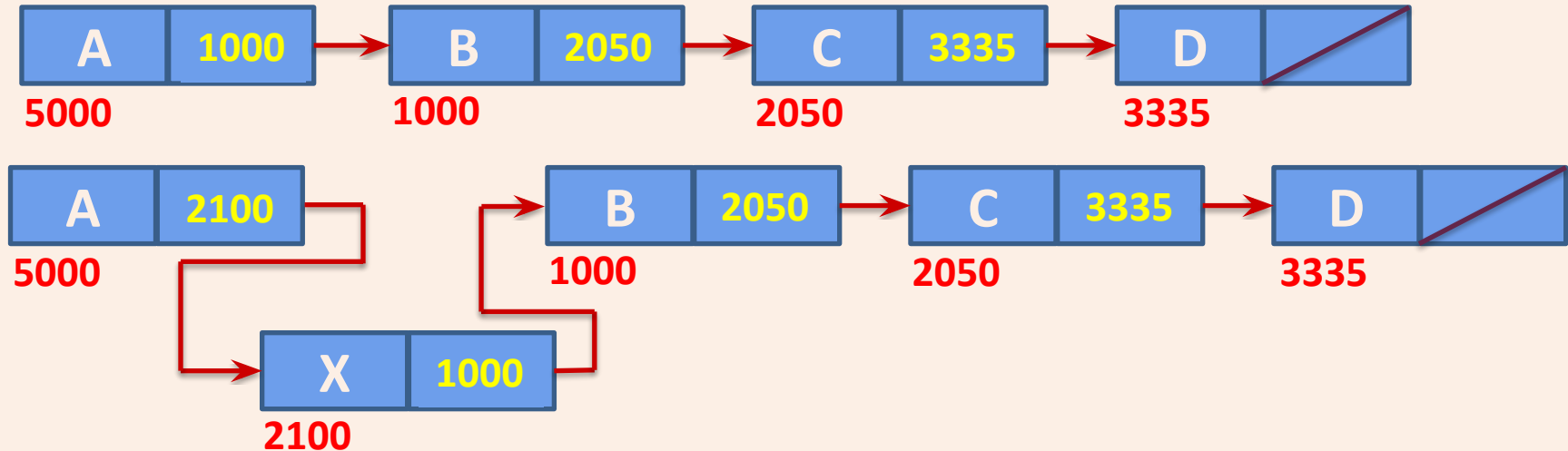


Singly Linked List

- There are several operations that are commonly performed on a singly linked list
 - Creation of List
 - Creation of Nodes
 - Insertion of Nodes into a List
 - Deletion of Nodes from a List
 - Traversing the Nodes in a List
- Search a Node in a List
- Join a List with another List
- Split a List into multiple List

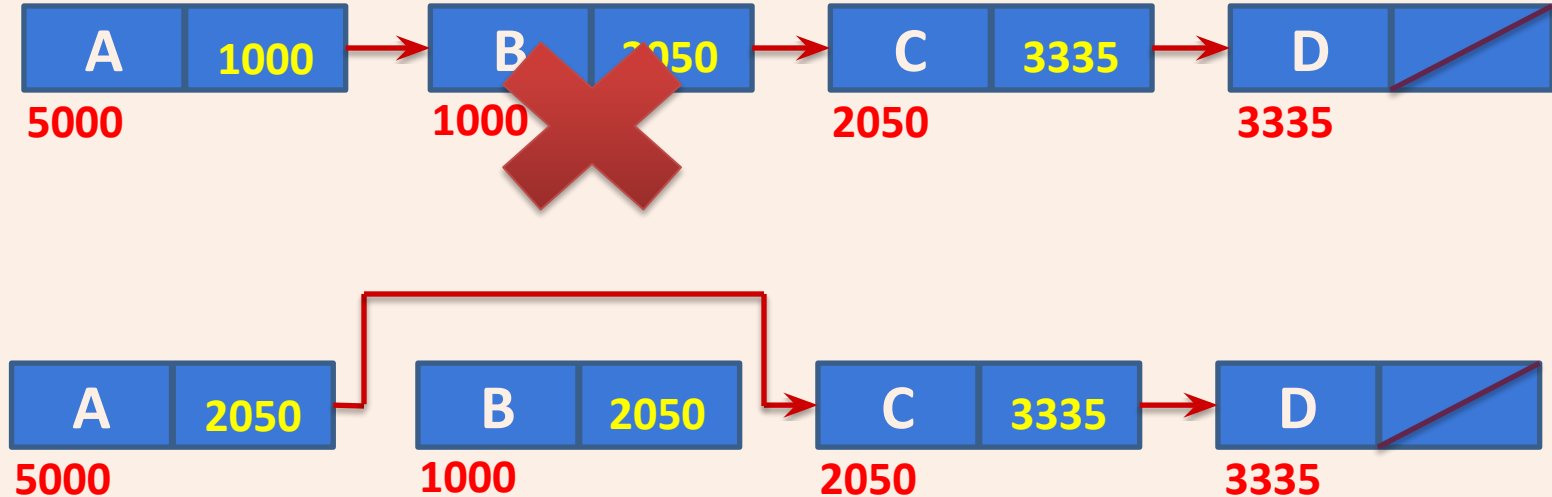
Singly Linked List - Insertion

- We have an n elements in list and it is required to insert a new element between the first and second element, what to do with sequential allocation & linked allocation?
- Insertion operation is more efficient in Linked allocation.



Singly Linked List - Deletion

- Deletion operation is more efficient in Linked Allocation



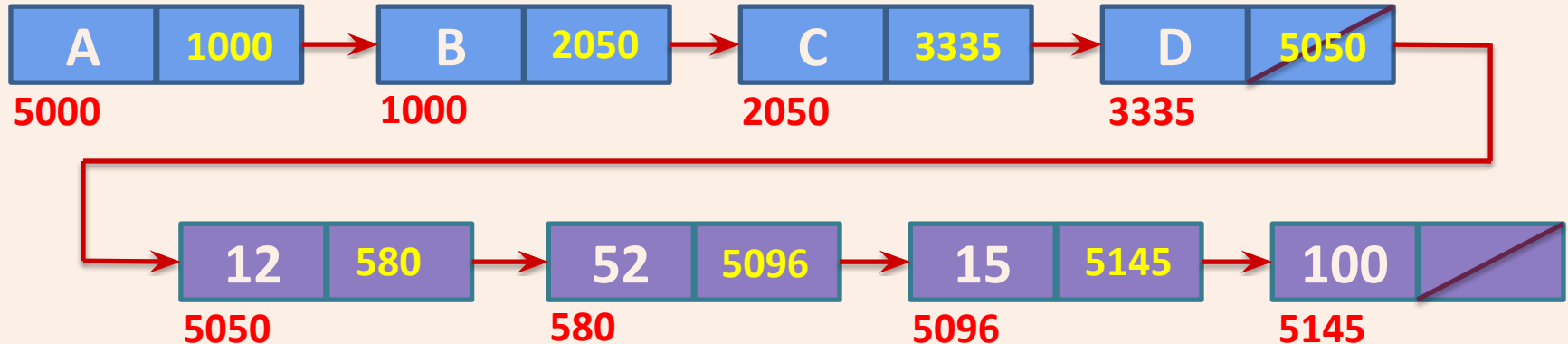
Singly Linked List - Search and Join

Search Operation

- **If particular node in the list is required**, it is necessary to follow links from the first node onwards until the desired node is found, in this situation **it is more time consuming** to go through linked list than a sequential list.
- Search operation is more time consuming in Linked Allocation.

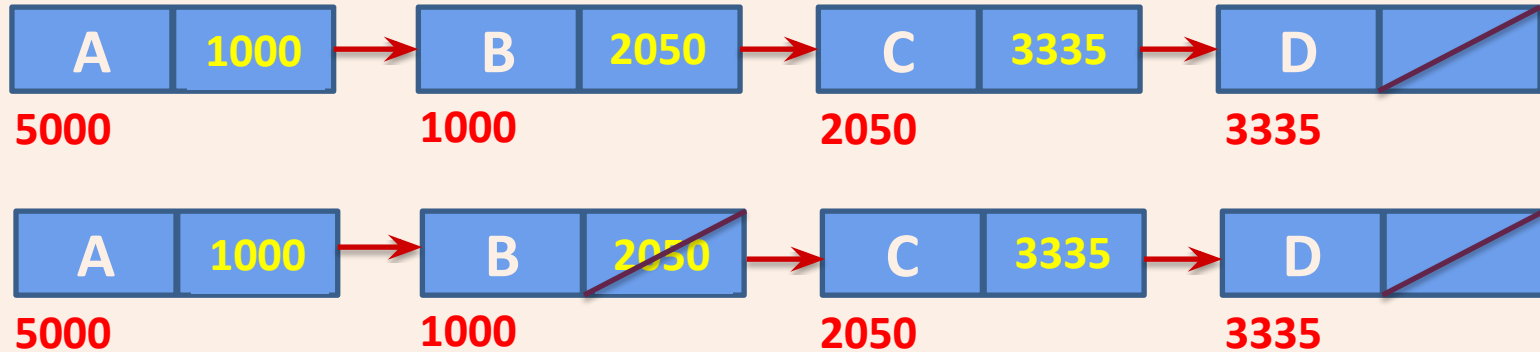
Join Operation

- Join operation is more efficient in Linked Allocation.



Singly Linked List - Split

- Split operation is more efficient in Linked Allocation



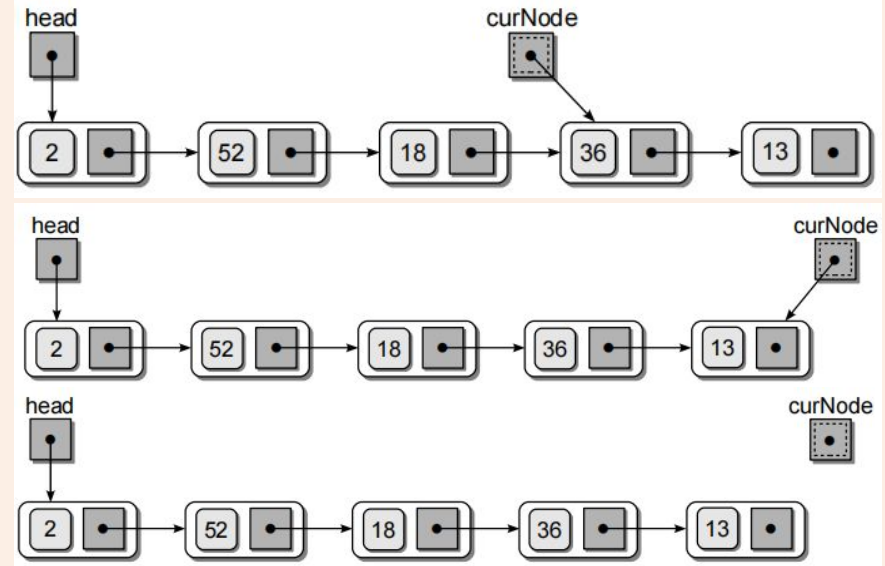
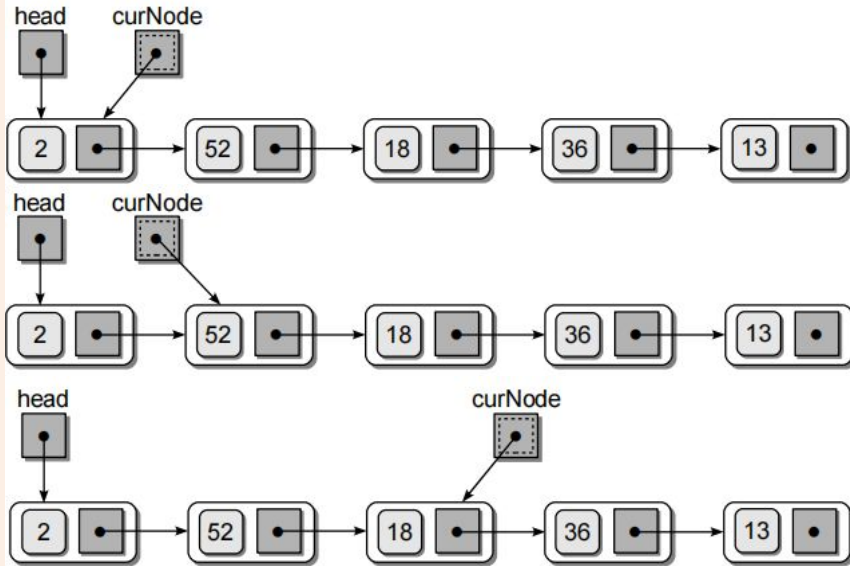
Singly Linked List - Traversing

```
def traversal( head ) :  
    curNode = head  
    while curNode is not None :  
        print curNode.data  
        curNode = curNode.next
```

- A correct implementation of the linked list traversal must also handle the case where the list is empty.
- An empty list is indicated by a null head reference.
- A complete list traversal requires $O(n)$ time since each node must be accessed and each access only requires constant time.

Singly Linked List - Traversing

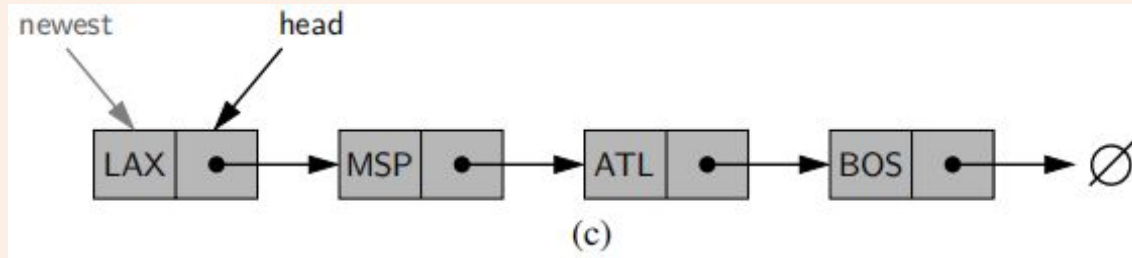
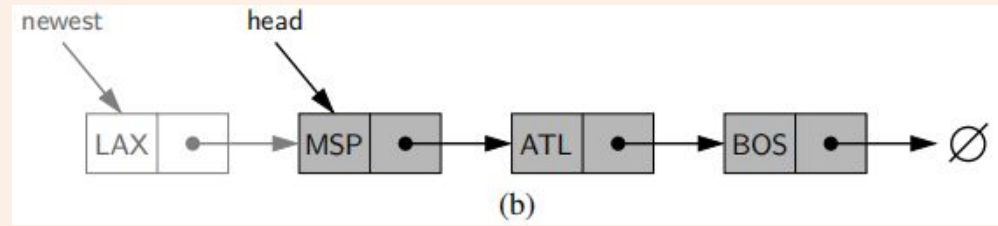
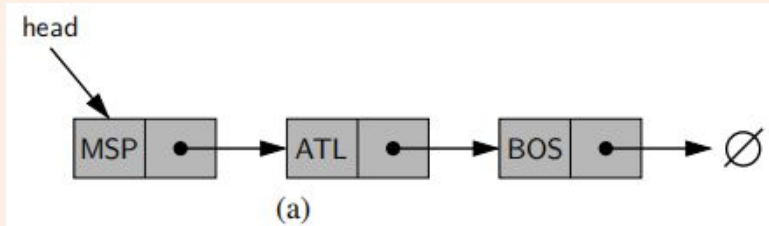
```
def traversal( head ):  
    curNode = head  
    while curNode is not None :  
        print curNode.data  
        curNode = curNode.next
```



SLL - Inserting at the Head

Algorithm `add_first(L,e)`:

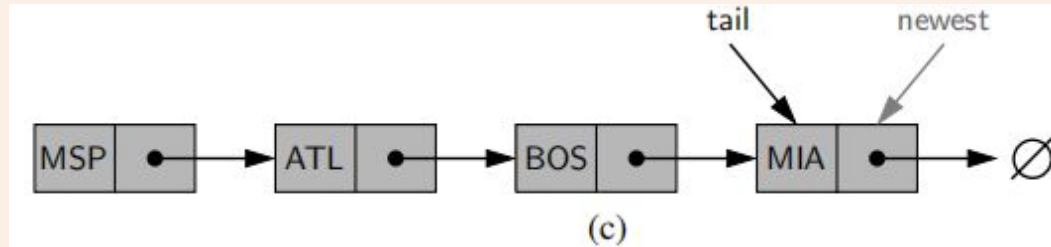
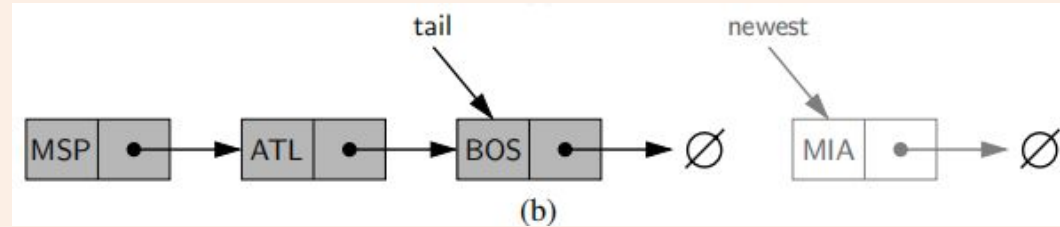
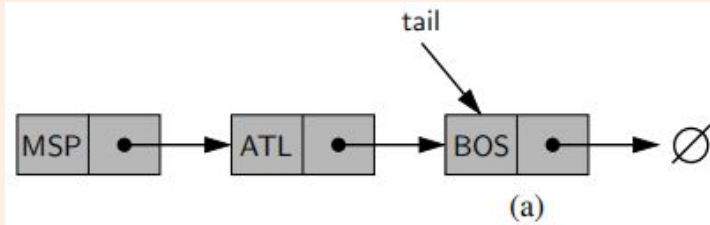
```
newest = Node(e) {create new node instance storing reference to element e}  
newest.next = L.head {set new node's next to reference the old head node}  
L.head = newest {set variable head to reference the new node}  
L.size = L.size+1 {increment the node count}
```



SLL - Inserting at the Tail

Algorithm `add_last(L,e)` :

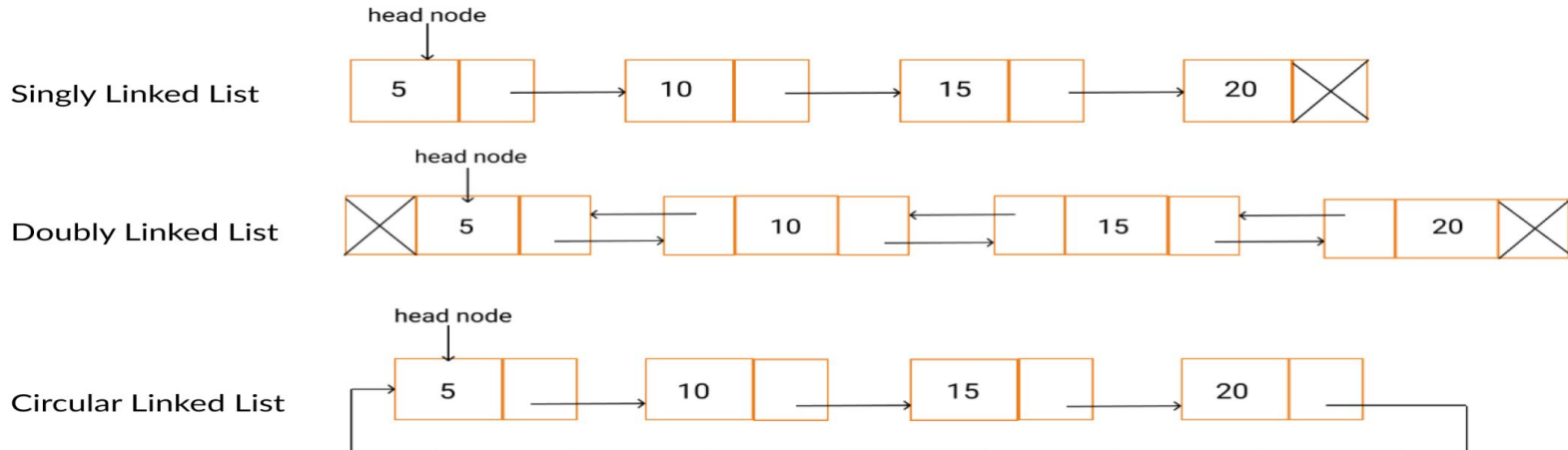
```
newest = Node(e) {create new node instance storing reference to element e}  
newest.next = None {set new node's next to reference the None object}  
L.tail.next = newest {make old tail node point to new node}  
L.tail = newest {set variable tail to reference the new node}  
L.size = L.size+1 {increment the node count}
```



Types of Linked List

- **Simple Linked List/Singly Linked List** – Item navigation is forward only.
- **Doubly Linked List** – Items can be navigated forward and backward.
- **Circular Linked List** – Last item contains link of the first element as next and the first element has a link to the last element as previous.

Types of Linked List



Operations on Linked List

	Operation	Description
1	Insertion at beginning	It involves inserting any element at the front of the list. We just need to a few link adjustments to make the new node as the head of the list.
2	Insertion at end of the list	It involves insertion at the last of the linked list. The new node can be inserted as the only node in the list or it can be inserted as the last one. Different logics are implemented in each scenario.
3	Insertion after specified node	It involves insertion after the specified node of the linked list. We need to skip the desired number of nodes in order to reach the node after which the new node will be inserted. .

Operations on Linked List

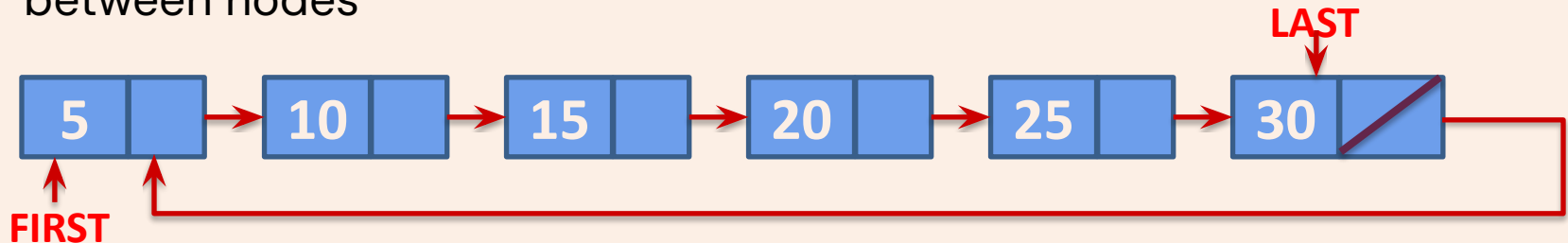
	Operation	Description
3	Deletion at beginning	It involves deletion of a node from the beginning of the list. This is the simplest operation among all. It just need a few adjustments in the node pointers.
4	Deletion at the end of the list	It involves deleting the last node of the list. The list can either be empty or full. Different logic is implemented for the different scenarios.
5	Deletion after specified node	It involves deleting the node after the specified node in the list. we need to skip the desired number of nodes to reach the node after which the node will be deleted. This requires traversing through the list.

Operations on Linked List

	Operation	Description
6	Traversing	In traversing, we simply visit each node of the list at least once in order to perform some specific operation on it, for example, printing data part of each node present in the list.
7	Searching	In searching, we match each element of the list with the given element. If the element is found on any of the location then location of that element is returned otherwise null is returned.

Circularly Linked Linear List

- If we **replace NULL** pointer of the **last node** of Singly Linked Linear List with the **address of its first node**, that list becomes circularly linked linear list or **Circular List**.
- **FIRST** is the address of first node of Circular List
- **LAST** is the address of the last node of Circular List
- **Advantages of Circular List**
 - In circular list, every node is accessible from given node
 - It saves time when we have to go to the first node from the last node. It can be done in single step because there is no need to traverse the in between nodes. But in double linked list, we will have to go through in between nodes



Circularly Linked Linear List

- **Disadvantages of Circular List**

- It is not easy to reverse the linked list
- If proper care is not taken, then the problem of infinite loop can occur
- If we at a node and go back to the previous node, then we can not do it in single step. Instead we have to complete the entire circle by going through the in between nodes and then we will reach the required node

- **Operations on Circular List**

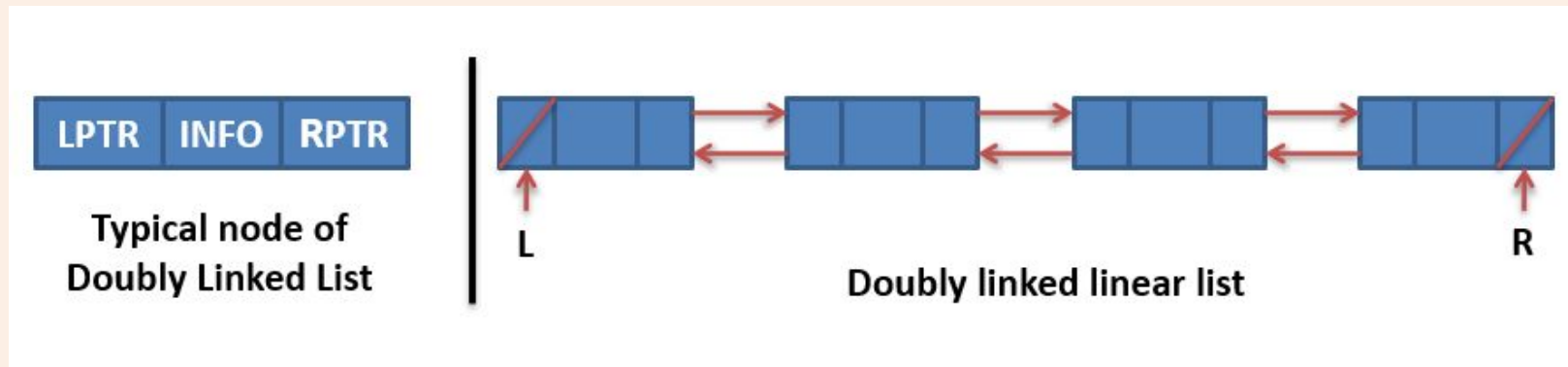
- Insert at First
- Insert at Last
- Insert in Ordered List
- Delete a node

Doubly Linked Linear List

- In certain Applications, it is very desirable that a list be traversed in either forward or reverse direction.
- This property implies that each node must contain two link fields instead of usual one.
- The links are used to denote **Predecessor** and **Successor** of node.
- The link denoting its **predecessor** is called **Left Link**.
- The link denoting its **successor** is called **Right Link**.
- A list containing this type of node is called **doubly linked list** or **two way chain**.

Doubly Linked Linear List

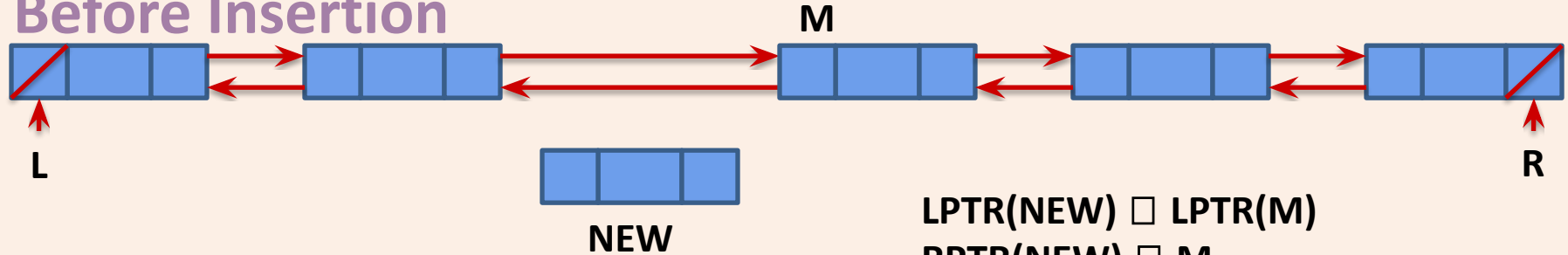
- Typical node of doubly linked linear list contains INFO, LPTR RPTR Fields
- **LPTR** is pointer variable pointing to Predecessor of a node
- **RPTR** is pointer variable pointing to Successor of a node
- Left most node of doubly linked linear list is called **L**, **LPTR** of node **L** is **always NULL**
- Right most node of doubly linked linear list is called **R**, **RPTR** of node **R** is **always NULL**



Insert node in Doubly Linked List

Insertion in the middle of Doubly Linked Linear List

Before Insertion



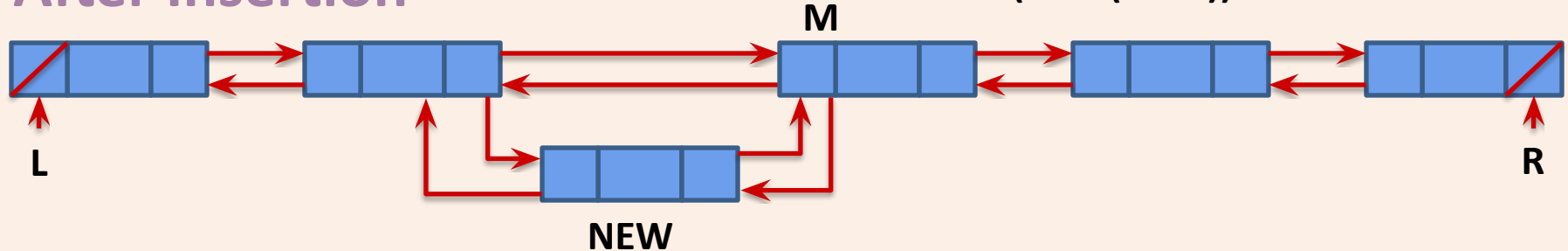
$LPTR(NEW) \square LPTR(M)$

$RPTR(NEW) \square M$

$LPTR(M) \square NEW$

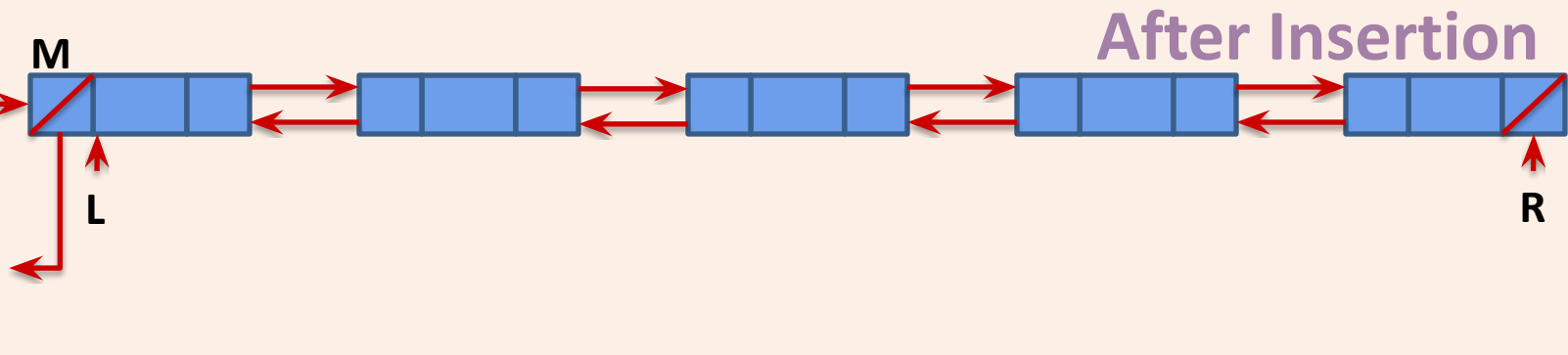
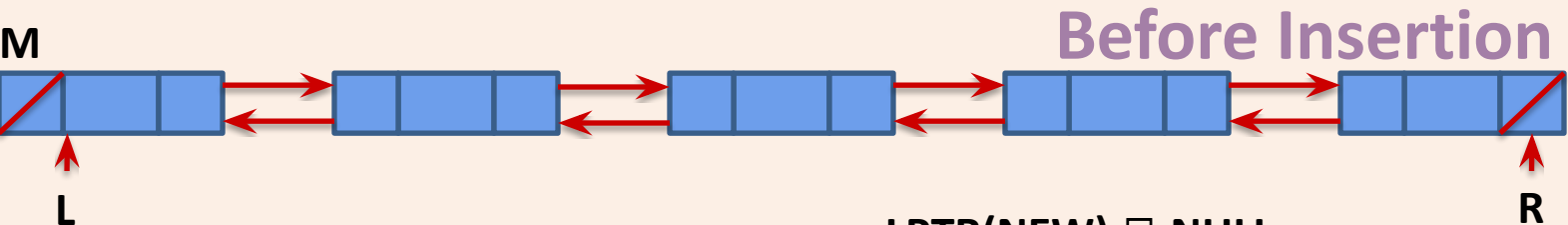
$RPTR(LPTR(NEW)) \square NEW$

After Insertion



Insert node in Doubly Linked List

Left most insertion in Doubly Linked Linear List



Implementation - Discussed in Class

- Singly Linked List
 - With Head Pointer
 - With Head and Tail Pointer
- Doubly Linked List
 - With Head Pointer
 - With Head and Tail Pointer
- Circular Singly Linked List
 - With Head Pointer
 - With Head and Tail Pointer
 - With Tail Pointer
- Doubly Circular Linked List (Possible Variation)



Thank You